

前期学习

学习顺序

先按照下面的聊天记录和notion上面的内容，把**isaacclab**的**框架**搞清楚

期间也可以用宇树的unitree_rl_lab跑g1或者go2的例程，看看有什么问题，宇树自己写的奖励有些实现的有问题

之后就**看rsl_rl**这个算法库，克隆下来,把distillation和ppo弄清楚

文档里面cartpole的例子是direct base，但我们现在大部分都用manager based。Direct based就是需要自己管理mdp五元组，每一步都要用单独的函数去读取机器人和环境相关的数据来计算obs和奖励，课程和随机化也比较麻烦。Manager base对于mdp五元组每一项都创建了一个管理器，这样可维护性和复用性都有提升。

ISAACLAB

source

isaacrab

docs

isaacrab

actuators

__init__.py

actuator_base_cfg.py

actuator_base.py

actuator_cfg.py

actuator_net_cfg.py

actuator_net.py

actuator_pd_cfg.py

actuator_pd.py

app

assets

articulation

deformable_object

rigid_object

rigid_object_collection

surface_gripper

utils

__init__.py

asset_base_cfg.py

asset_base.py

controllers

devices

envs

mdp

actions

commands

recorders

执行器

机器人数据结构

MDP 5 tuple

source > isaaclab > isaaclab

28 class JointAct

60 def __init

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

Properties

"""

Mayank

@property

def action

return

@property

def raw_a

return

@property

def proce

return

@property

def IO_de

"""Th

This

MDP 3-tuple
xxx_func

- __init__.py
- curriculum.py
- events.py
- observations.py
- rewards.py
- terminations.py

> ui

> utils

__init__.py

common.py

direct_marl_env_cfg.py

direct_marl_env.py

direct_rl_env_cfg.py

direct_rl_env.py

仿真流程

manager_based_env_cfg.py

manager_based_env.py

manager_based_rl_env_cfg.py

manager_based_rl_env.py

manager_based_rl_mimic_env.py

mimic_env_cfg.py

managers

__init__.py

action_manager.py

command_manager.py

curriculum_manager.py

event_manager.py

manager_base.py

manager_term_cfg.py

observation_manager.py

recorder_manager.py

reward_manager.py

scene_entity_cfg.py

termination_manager.py

> markers

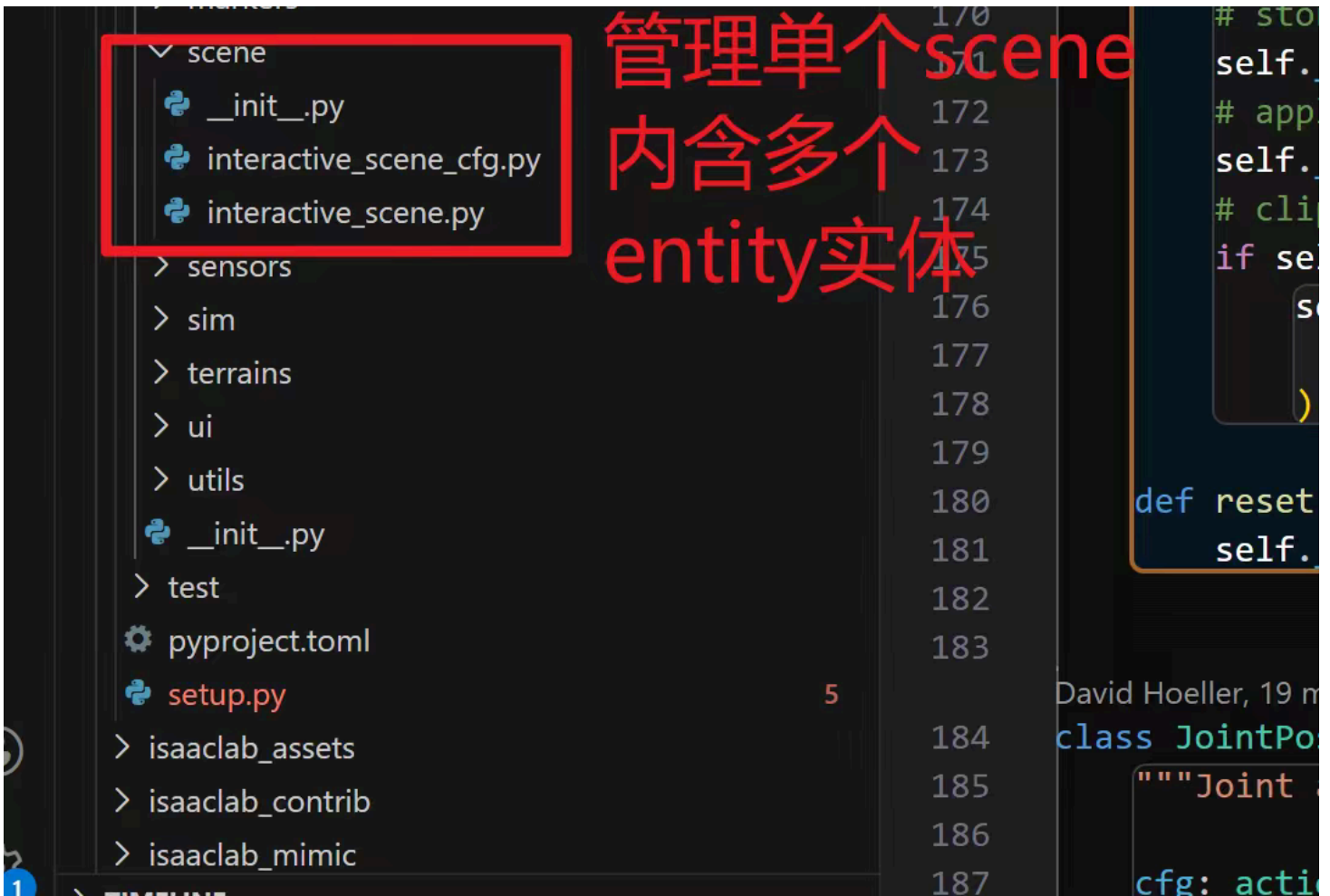
call

mdp.xxx_func
调用处理

135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169

It ad
- join
- sca
- off
- cli

Return
T
""
super
self.
self.
self.
self.
self.
self.
Thi
if is
s
else:
s
Delegate to
FIX
if se
i
e
else:
s
return
""
Operation
""
def proce



Notion link

https://lapis-router-502.notion.site/hcl-lab-kickstart?source=copy_link

聊天记录

自己把代码看一遍，从使用cfg进行初始化，到每一步仿真中的数据流调用：

- 比如你现在想随机化每个关节的干摩擦系数，你要怎么修改关系；
- 再比如你要创建一个考虑了电机反电动势和轴向磁通效应的电机模型，且你的控制量是速度，那你要怎么添加jointAction

这个要在EventCfg里面修改

是在这里，但是怎么访问prim的这个属性？ prim path的树状结构是咋样的，一个机器人是用啥对象描述，articulation，rigid object collection，以及rigid body、joint这些属性怎么获取

关键两件事，1是知道仿真中初始化以及步进中的所有数据流和调用关系，以及如何处理termination和reset；2是熟悉仿真中机器人对象以及其他环境中物体的数据结构和api，也就是想要访问机器人下的数据、进行修改等要怎么操作

比如termcfg里面传入一个mdp.term_func，term_func会根据函数里的判断条件分析是否应该终止。但这个term_func具体是哪里调用的？调用频率多少？被谁调用？和奖励计算、观测计算的先后关系是什么

同理也可以推广到action、obs、event还有curriculum

本质上就是mdp五元组+event+curriculum

为了计算奖励、观测，判断是否中止之类的，就需要获取机器人和环境相关的数据，所以刚刚和你说，要把如何访问env以及其中的articulation（主要是机器人）以及刚体等弄清楚

那么为了应用控制或者施加外力扰动，就需要将一些数据写入机器人，或者写入仿真器

比如，下面是我让claudecode浏览isaacclab以后总结的：

场景中Prim的层级结构和表示方式

1. 场景层级结构

在Isaac Lab中，场景的prim层级结构遵循以下模式：

以及MDP里各个manager的处理流程：

Isaac Lab Manager-based 环境 MDP 全流程分析

2. 环境初始化流程

1.1 核心组件初始化

ManagerBasedRLEnv 继承自 ManagerBasedEnv 和 gym.Env，初始化流程如下：

之后如果上地形，最好再把terrain generator和terrain管理看明白

你还要看看rsl rl的源码，这是unitree rl lab默认的rl library。看看isaacclab是怎么实现vec_env的

后续要做mimic任务的话，就按照类似的流程看isaacclab_mimic部分的代码。场景里的实体（机器人、其他可交互物体）是一样的，但是mimic的逻辑和纯rl训练有些差异。

你看下mdp里面随便一个和robot数据相关的函数是怎么访问robot数据的:隔了15天后的回复

任务实现

1、基本任务

基于当前的躯干+下肢urdf，训练amp行走/奔跑+mimic舞蹈/行走/奔跑

0. 当前urdf是串联。

1. urdf有一些小问题，你查一下然后修复，并给我反馈。一般推荐对称关节做镜像动作的时候是角度变化方向相同，但joint坐标不改也可以。bonus: 其他问题查出来，再告诉我[旺柴] 有些问题影响训练性能和速度。
2. 纯RL任务实现串联行走，并在mujoco部署，4m/s速度跟踪

3. retarget LAFAN动作到test_robo上，实现至少3个序列的mimic任务。可以单序列过拟合，也可以训练一个unify wbc tracker。
4. 基于2，利用LAFAN retarget的行走和奔跑序列做amp-RL训练，实现拟人步态的运动
5. 创建并联mjcf文件，使用串联训练，但在mujoco中进行串转并部署（g1实际上内部完成了该转换，它也支持ab mode，直接控制小腿上驱动踝关节的电机，就需要自己进行fk&ik）
6. bonus: 自己创建并联的USD/mjcf，直接用 并联构型 在isaacsim/mjlab中完成上述训练

2、进阶任务

1. amp可以实现明显的风格化行走，你把lafan里面的行走、奔跑、冲刺都加进来，需要加一些trick让策略可以在不同速度的情况下平滑的过渡。
2. mimic要弄完，参考beyondmimic (whole_body_tracking)或者unitree_rL_lab的mimic改，这个所需的retarget数据和amp是类似的，至少实现行走奔跑冲刺的序列，也可以自己尝试舞蹈（但没有手所以看不太出来）

3、参考资料

- 有些仓库用的gym，但是rl library都是可移植的。可以先摸透**manager/direct based**的训练框架以及lab是如何与rl library (e.g. rsl rl) 对接的方式：<https://lapis-router-502.notion.site/2aca501c6ee58049b8d2da6fd006d6c7>
- 机器人描述文件（urdf/mjcf/usd）在线查看：
<https://viewer.robotsfan.com/>
<https://gkjohnson.github.io/urdf-loaders/javascript/example/bundle/>
mujoco viewer（按i查看惯量，按1/2/3/4显示碰撞体、mesh，按t透明mesh/碰撞体）
- 参考仓库：
 - i. retarget: <https://github.com/amazon-far/holosoma> <https://github.com/YanjieZe/GMR>
 - ii. amp:
 - https://github.com/linden713/humanoid_amp
 - <https://github.com/escontra/>
 - <https://github.com/gbionics/amp-rsl-rl/tree/main>
 - iii. AMP_for_hardware <https://github.com/Open-X-Humanoid/TienKung-Lab>
 - iv. RL & tracking codebase:
 - https://github.com/unitreerobotics/unitree_rL_lab
 - https://github.com/HybridRobotics/whole_body_tracking
 - <https://github.com/xbpeng/MimicKit>
 - isaacsim自带的 tracking task

- **如何使用coding agent**

建议写代码多用原生ide/cli coding agent哈，学习怎么与ai交互、怎么提需求，怎么让它教你看代码。有问题多和llm讨论。闲鱼上可以开gpt plus/team，一个月十多块。也可以用国产qoder/trae有一些免费额度。github-copilot验证学生身份之后可以直接使用

- cs146s 现代软件开发 : <https://themodernsoftware.dev/> ****
- 如何使用mcp和skill: <https://anthropic.skilljar.com/claude-with-the-anthropic-api>
- cursor official, how to use agent: <https://cursor.com/cn/learn>

- **算力资源**

<https://www.gpufree.cn/home>

<https://www.autodl.com/login>

这里有一些免费资源可用，阿里和火山也有赠送isaacclab开箱即用的实例。